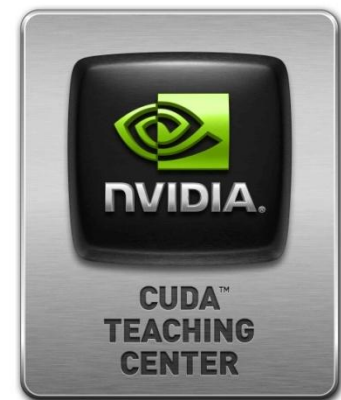


Programación masivamente paralela en procesadores gráficos (GPUs)

E. Dufrechou, M. Freire, P. Ezzatti y M. Pedemonte



Clase 4

Arquitectura de la GPU

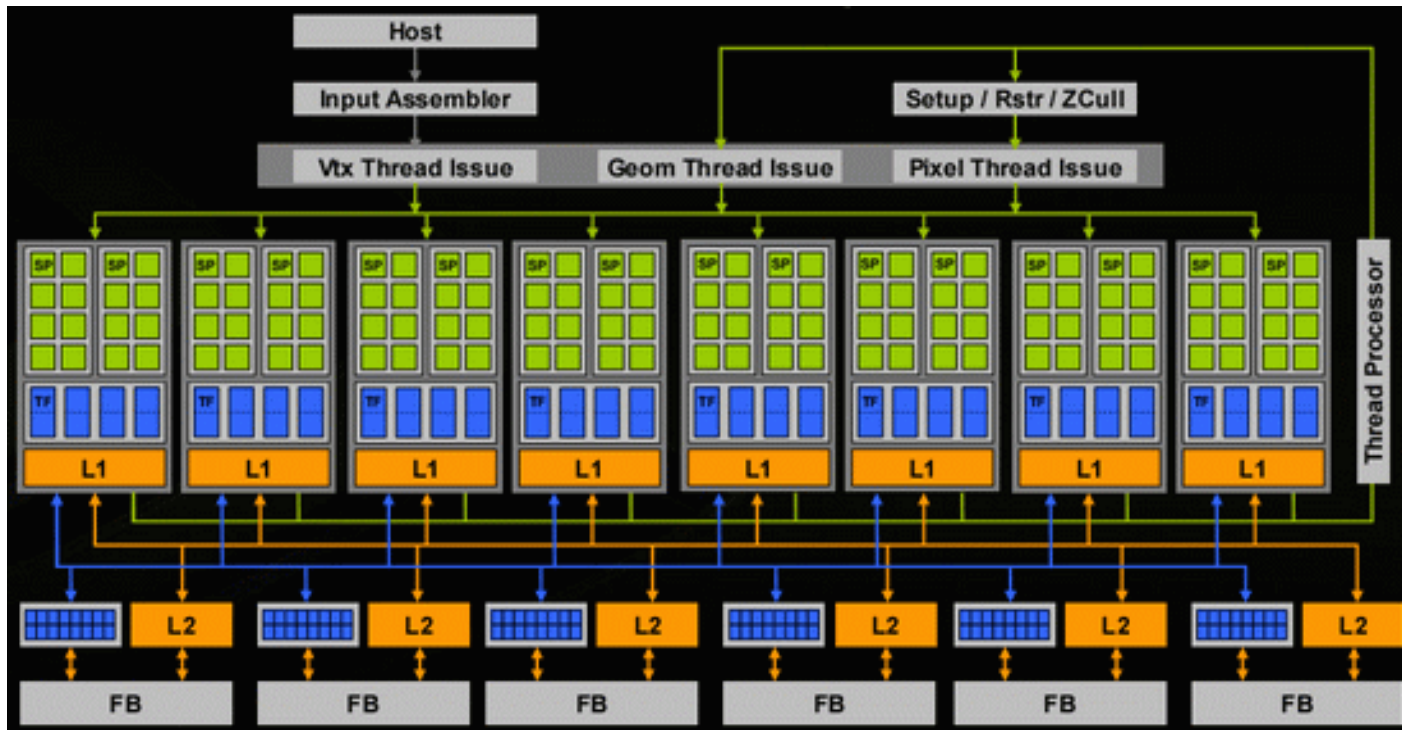
Contenido

- **Arquitectura CUDA – G80**
- **Modelo de Ejecución Clásico**
- **Jerarquía de Memoria Clásica**
- **Arquitectura Fermi**
- **Arquitectura Kepler**
- **Arquitectura Maxwell**
- **Arquitectura Pascal**
- **Arquitectura Volta**
- **Arquitectura Turing**
- **Compute Capabilities**

Arquitectura CUDA – G80

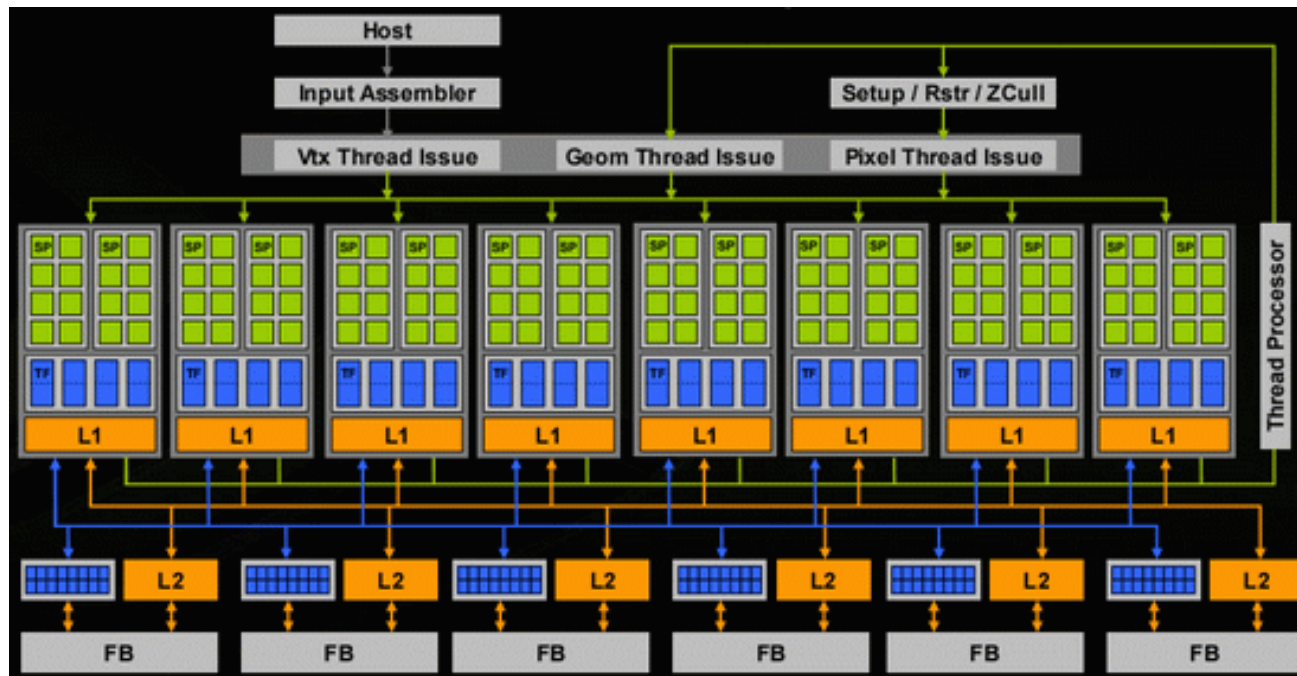
Arquitectura CUDA – G80

La Arquitectura G80



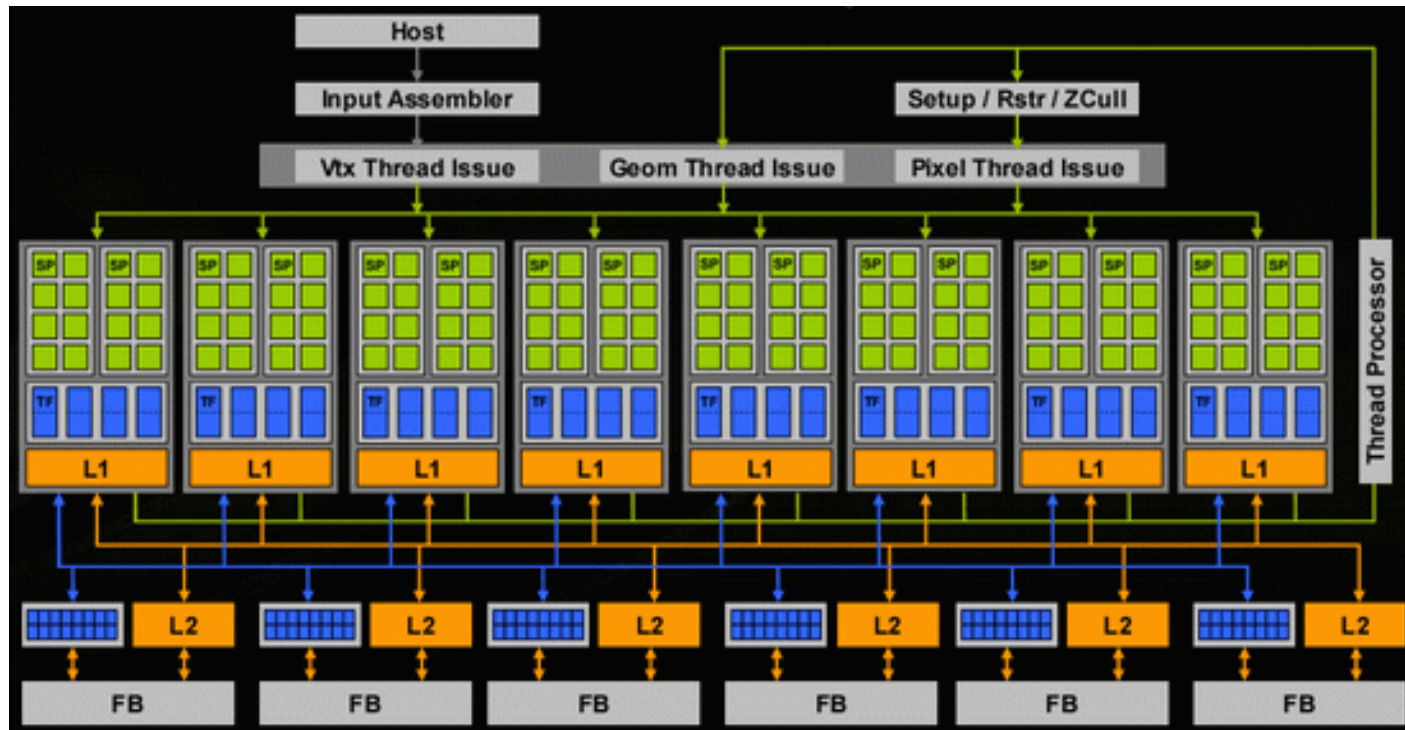
Arquitectura CUDA – G80

- La arquitectura CUDA puede verse como un arreglo escalable de multiprocesadores.
- SMs: multithreaded Streaming Multiprocessors.
- La arquitectura de la G80 (primera de CUDA) tiene 16 SMs.



Arquitectura CUDA – G80

- Los SMs se agrupan en pares formando 8 Thread Processing Clusters (TPCs).
- Los TPCs contienen los caches de texturas y de memoria constante que son compartidos por los SMs.



Arquitectura CUDA – G80

- **Un multiprocesador (SM) de la arquitectura G80 consiste en:**
 - **ocho procesadores escalares (Streaming Processors, SP). También conocidos como CUDA cores.**
 - **unidad de instrucciones multihilo**
 - **chip de memoria compartida**
 - **dos unidades especiales (Special Function Units, SFU) para computar operaciones trascendentales como sin, cos, log, y sqrt.**

Arquitectura CUDA – G80

- Cada CUDA core tiene una unidad que permite realizar una operación multiply-add y una unidad que permite hacer una multiplicación.
- Es decir que los CUDA cores básicamente son una ALU.
- Los CUDA cores no tienen registros propios o caches.
- Los registros se manejan a nivel de multiprocesador.
- Los CUDA cores soportan la aritmética de punto flotante de simple precisión (32 bits) definida en el estándar IEEE 754-1985.

Arquitectura CUDA – G80

- La arquitectura de la CPU utiliza caché de datos para ocultar la latencia en el acceso a la memoria RAM.
- La arquitectura de la GPU elimina (o reduce sensiblemente) el caché de datos.
- Las GPUs para ocultar la latencia en el acceso a la memoria:
 - cuando un conjunto de threads accede a memoria, se suspende su ejecución hasta que los datos hayan llegado al SM.
 - otro conjunto de threads comienza a ejecutar.
 - el scheduling (planificación) se realiza por hardware en forma transparente para el programador y prácticamente sin costo.
- Esto permite tener muchos más threads que unidades de ejecución.

Arquitectura de CUDA – G80

- Los multiprocesadores tienen la capacidad de crear, gestionar y ejecutar hilos concurrentemente prácticamente sin costo de planificación.
- Los SMs de la arquitectura G80 pueden ejecutar hasta 768 hilos concurrentemente pero no al mismo momento.
- Por lo tanto, la tarjeta puede ejecutar 12288 hilos concurrentemente!!!

Modelo de Ejecución Clásico

Modelo de ejecución clásico

- La arquitectura CUDA permite abstraer la GPU como un conjunto de multiprocesadores compuestos a su vez por un conjunto de procesadores orientados a la ejecución de hilos.
- En el modelo de ejecución de CUDA cada multiprocesador ejecuta el mismo programa pero sobre distintos datos.
- Los distintos multiprocesadores **NO** tiene que estar ejecutando la misma instrucción en el mismo momento.
- Este paradigma de programación se conoce con el nombre SPMD (single program multiple data).

Modelo de ejecución clásico

- Internamente cada multiprocesador sigue el paradigma de programación paralela SIMT (Single Instruction Multiple Threads).
- SIMT es similar a SIMD (Single Instruction Multiple Data) ya que una sola instrucción controla múltiples elementos de procesamiento.
- Sin embargo, no son exactamente la misma cosa.

Modelo de ejecución clásico

- **SIMD (como las vector machines o Intel MMX y SSE):**
 - Existe una única unidad de procesamiento que ejecuta secuencialmente instrucciones.
 - Algunas instrucciones son escalares y otras vectoriales, pero en ese caso es una instrucción operando sobre varios elementos de datos.

Modelo de ejecución clásico

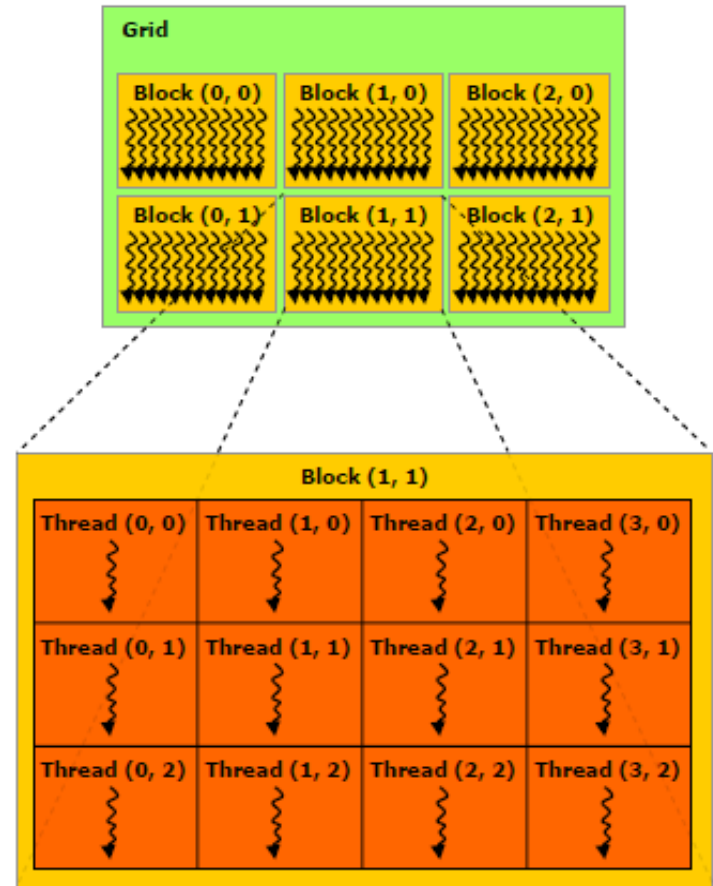
- **SIMT (como las GPUs):**
 - Cada “carril” del cómputo vectorial puede verse como un hilo separado.
 - Existe una única unidad (hardware sequencer) que se encarga de gestionar el trabajo en paralelo sobre un grupo de esos hilos.
 - Una única instrucción se difunde a todos los elementos de procesamiento.
 - Si todos los hilos deben ejecutar la misma instrucción, se dice que los hilos mantienen coherencia.
 - Si los hilos deben ejecutar instrucciones distintas, los hilos divergen.
 - La unidad (hardware sequencer) mantiene la información de que hilos han divergido.

Modelo de ejecución clásico

- **SIMT (como las GPUs):**
 - La unidad difunde una única instrucción por ciclo a todos los elementos de procesamiento que pueden ejecutar (en función de las divergencias). Algunos hilos se deshabilitan (los que divergieron).
- Como consecuencia del funcionamiento descrito, el mismo programa podría ejecutar si cada hilo ejecutara en forma independiente en un core distinto.
- En contraste con SIMD, SIMT permite tener:
 - paralelismo a nivel de datos: cuando los hilos son coherentes.
 - paralelismo a nivel de hilo: cuando los hilos divergen, cada hilo ejecuta en forma independiente.

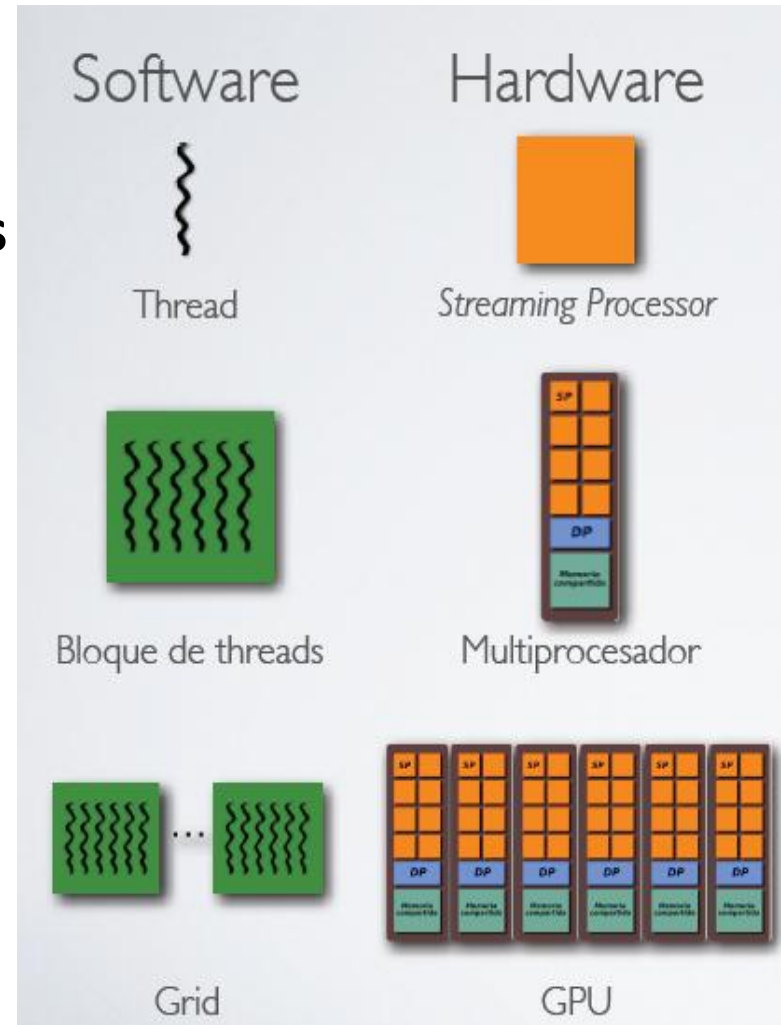
Modelo de ejecución clásico

- Los programas que ejecutan en la GPU se denominan kernels.
- La GPU puede ejecutar un kernel a la vez.
- La ejecución de un kernel es realizada por hilos que se organizan en bloques.
- Los bloques pueden ser unidimensionales, bidimensionales o tridimensionales.
- Los bloques se organizan en un grid.
- El grid puede ser unidimensional o bidimensional.



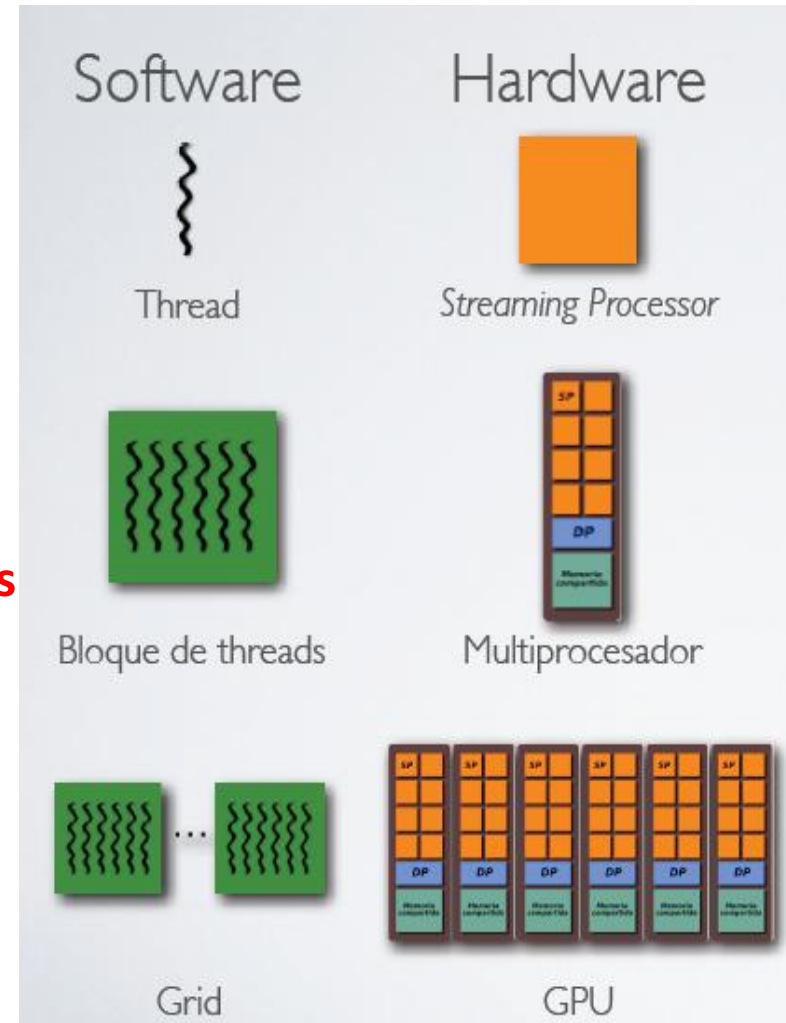
Modelo de ejecución clásico

- Cada **bloque** es un subconjunto de los cálculos que será ejecutado en un multiprocesador (MP) en forma **independiente**.
- Si hay suficientes MPs disponibles, todos los bloques de un grid son ejecutados en paralelo.



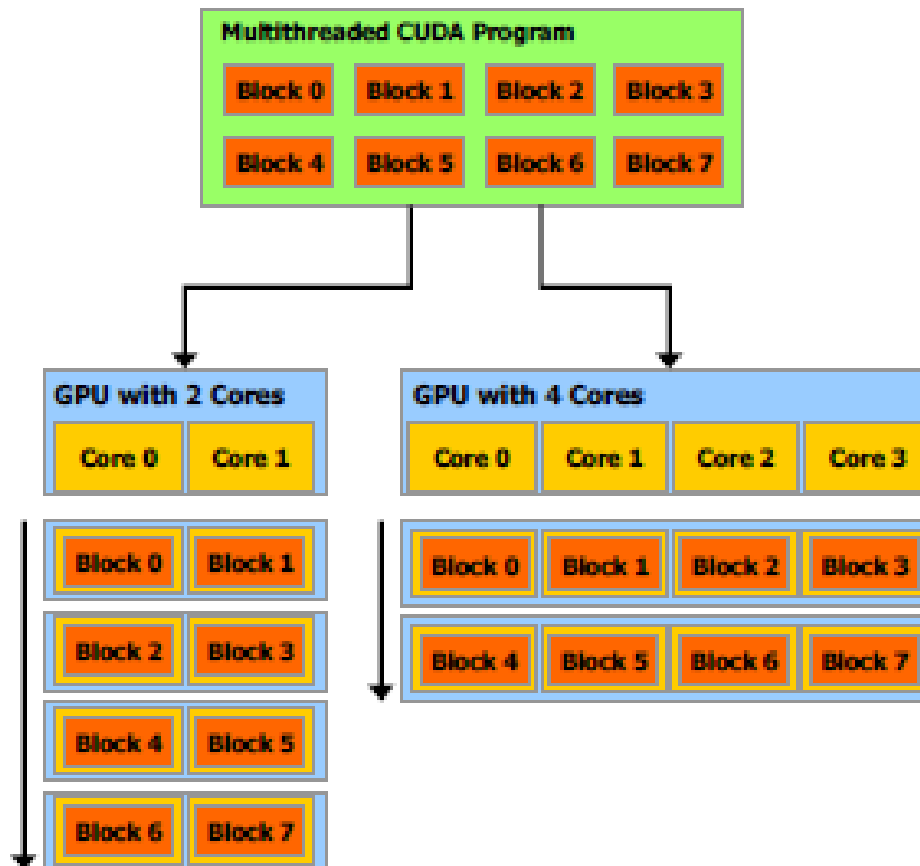
Modelo de ejecución clásico

- En caso contrario, el scheduler gestiona la ejecuciones de los bloques:
 - Un MP puede ejecutar múltiples bloques.
 - El **orden de ejecución** de los bloques **no es conocido a priori**.
- Cada hilo de un bloque ejecuta en un CUDA core del MP.



Modelo de ejecución clásico

- Organizar la ejecución de esta manera permite escalar automáticamente.



Modelo de ejecución clásico

- La ejecución se planifica en base a warps.
- El MP crea, gestiona, planifica y ejecuta en base a grupos de hilos llamados warps.
- El tamaño de warp en la arquitectura G80 es 32 hilos.
- Cuando un MP debe ejecutar un bloque, lo particiona en warps.
- Cada warp contiene hilos consecutivos.

Modelo de ejecución clásico

- Un warp ejecuta una instrucción a la vez.
- Si los hilos de un warp divergen debido a una condición de bifurcación dependiente de los datos:
 - El warp serializa la ejecución de cada camino de la bifurcación, deshabilitando los hilos que no forman parte del camino de ejecución.
 - Cuando todos los caminos completan su ejecución, los hilos convergen al mismo camino de ejecución.
- Como consecuencia, la eficiencia máxima se logra cuando los hilos de un warp coinciden en su camino de ejecución.
- La divergencia ocurre únicamente dentro de un warp.
- Distintos warps ejecutan en forma independiente, sin tener en cuenta si están ejecutando caminos comunes o disjuntos.

Jerarquía de Memoria Clásica

Jerarquía de memoria

- Existen varios tipos de memoria distintos en el dispositivo:
 - Global
 - Local
 - Compartida
 - Registros
 - Constante
 - Texturas
- Diremos que una memoria es on-chip cuando está en el mismo chip que los CUDA cores.
- Diremos que una memoria es off-chip cuando **NO** está en el mismo chip que los CUDA cores.

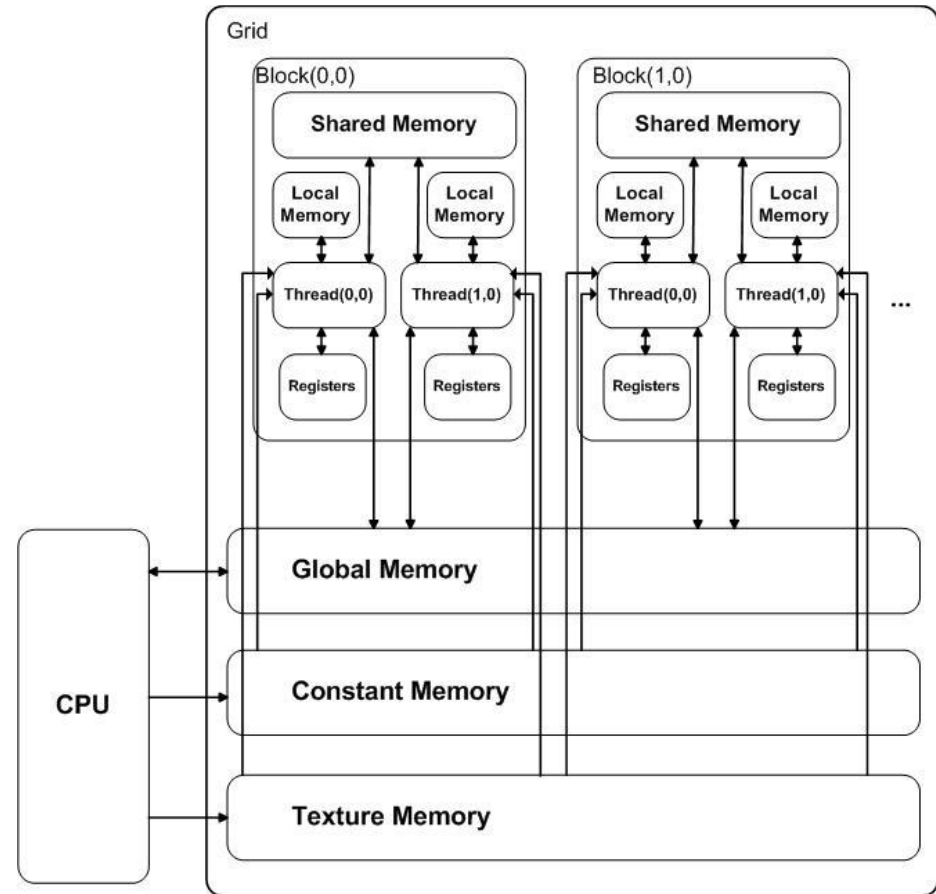
Jerarquía de memoria

- **Memoria global:**

- Todos los hilos que ejecutan en la GPU tienen acceso al mismo espacio global de memoria.
- Es off-chip.
- El acceso es el más lento y no es cacheado.

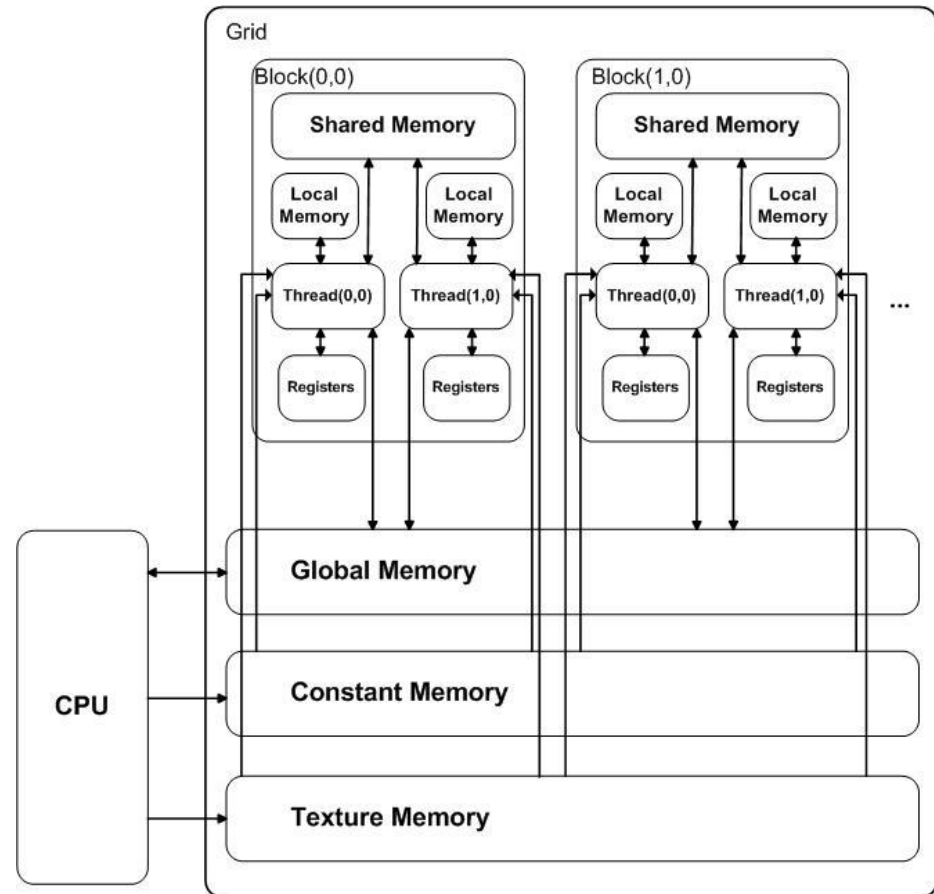
- **Registros:**

- Es on-chip.
- Es la más rápida de la GPU.
- Sólo son accesible por cada hilo.
- Este espacio de memoria es gestionada por el compilador.



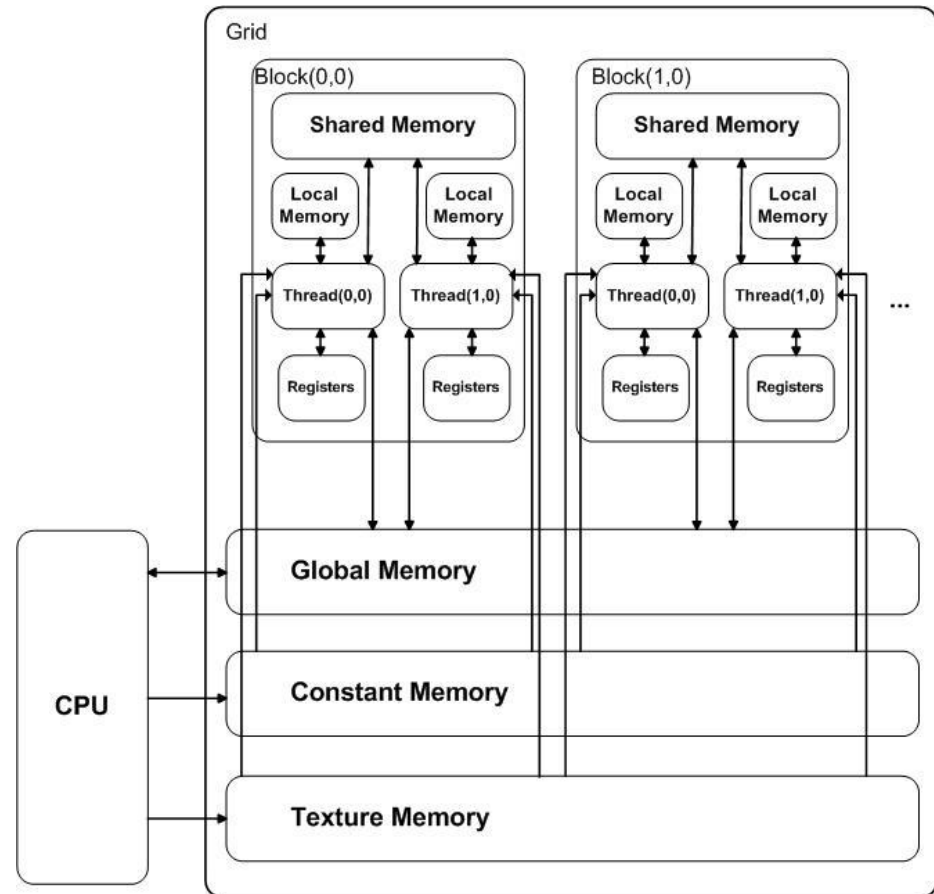
Jerarquía de memoria

- **Memoria local:**
 - Cada hilo tiene su propia memoria local.
 - Es una de las memorias más lentas de la GPU y no es cacheado.
 - Es off-chip.
 - Este espacio de memoria es gestionada por el compilador.
 - La figura muestra esta memoria cercana a los hilos y como privada a cada hilo. Sin embargo, realmente es off-chip.



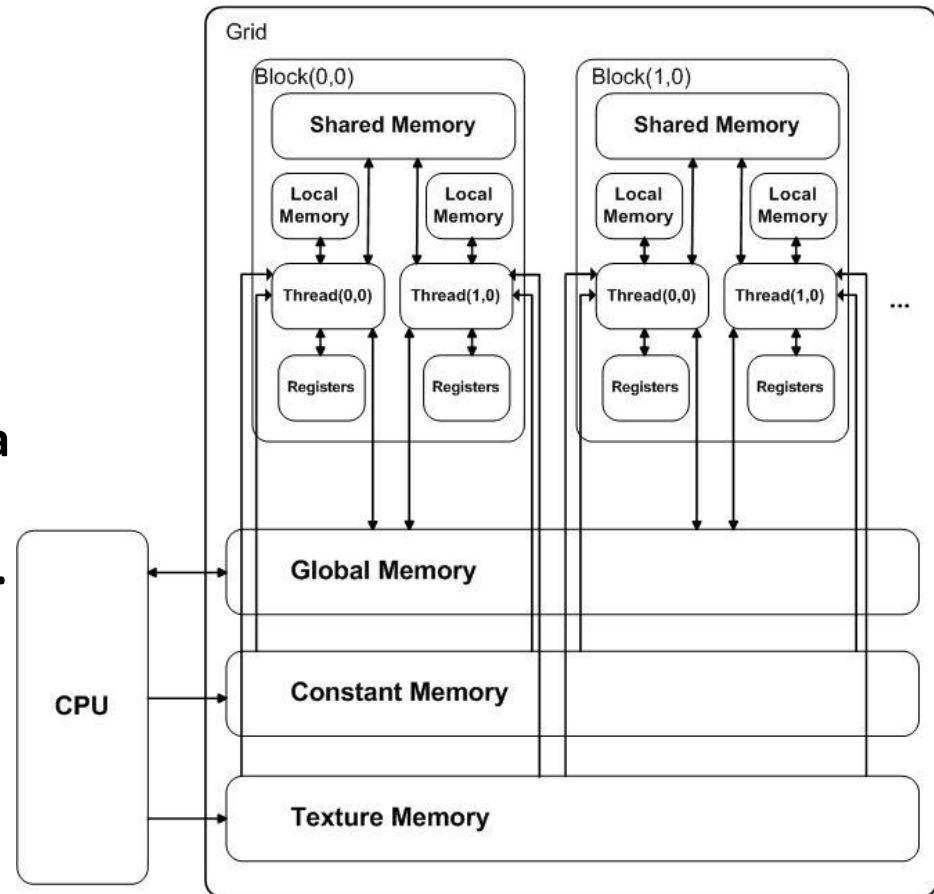
Jerarquía de memoria

- **Memoria compartida:**
 - Cada bloque tiene un espacio de memoria compartida que es prácticamente tan rápida como los registros.
 - Puede ser accedida por cualquier hilo del bloque.
 - Es on-chip.
 - Su tiempo de vida es igual al tiempo de vida del bloque.



Jerarquía de memoria

- **Memoria constante:**
 - Es una memoria rápida.
 - Para el dispositivo es solamente de lectura
 - Es off-chip aunque es cacheada.
 - Puede ser vista como un caché a memoria global más que como un espacio de memoria distinto.
- **Texturas:**
 - Tiene características similares a la memoria constante.

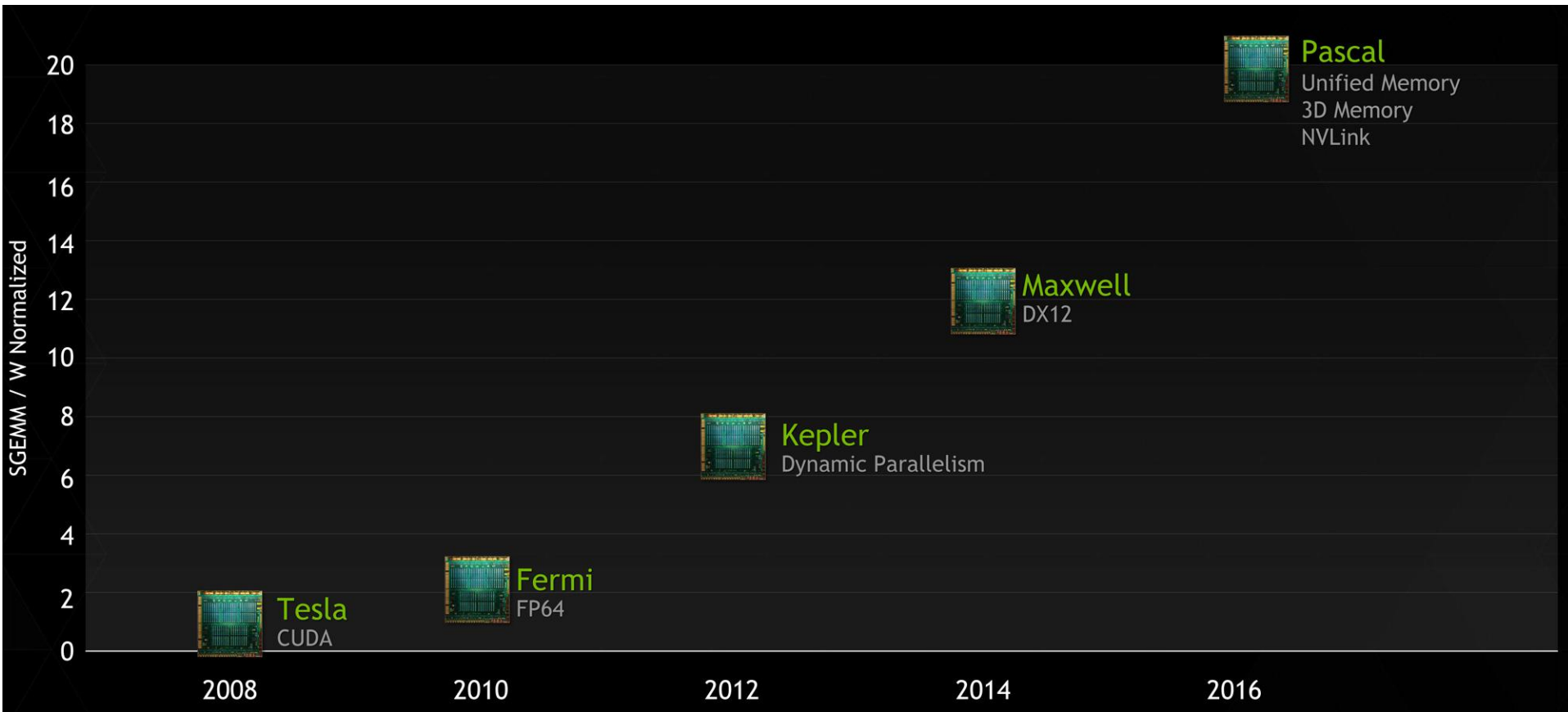


Jerarquía de memoria

Memoria	Ubicación	Caché	Acceso	Alcance	Existencia
Constante	Off-chip	Sí	R	Device	Aplicación
Local	Off-chip	No	R-W	Thread	Aplicación
Global	Off-chip	No	R-W	Device	Aplicación
Compartida	Chip	No	R-W	Bloque	Bloque
Registros	Chip	-	R-W	Thread	Thread
Texturas	Off-chip	Sí	R??R-W??	Device	Aplicación

Arquitectura Fermi

Arquitectura Fermi



Arquitectura Fermi

- En 2010 se lanza la segunda arquitectura de CUDA.
- Algunos de los cambios más importantes son:
 - Los CUDA cores soportan completamente el estándar 754-2008 de la IEEE para simple y doble precisión (se agregó el soporte para números desnormalizados).
 - Se unifica el espacio de direcciones de memoria (global, shared y local), lo que permite dar soporte completo a C++.

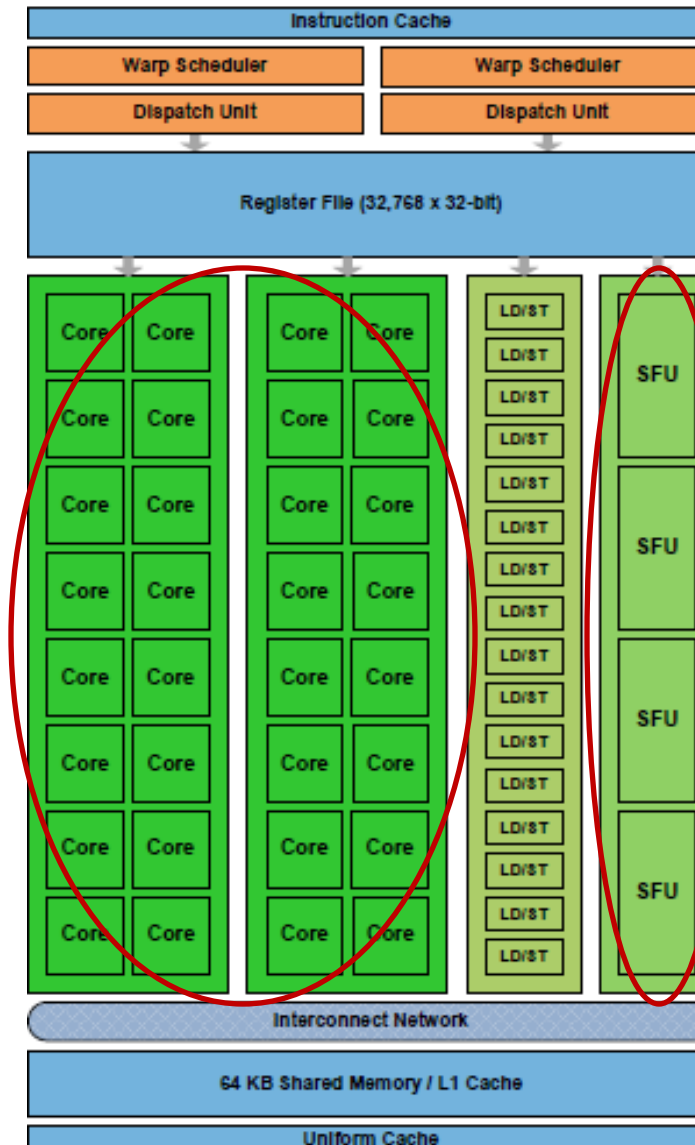
Arquitectura Fermi

- Algunos de los cambios más importantes son:
 - Tiene 16 SMs con 32 CUDA cores cada uno (512 cores en total).



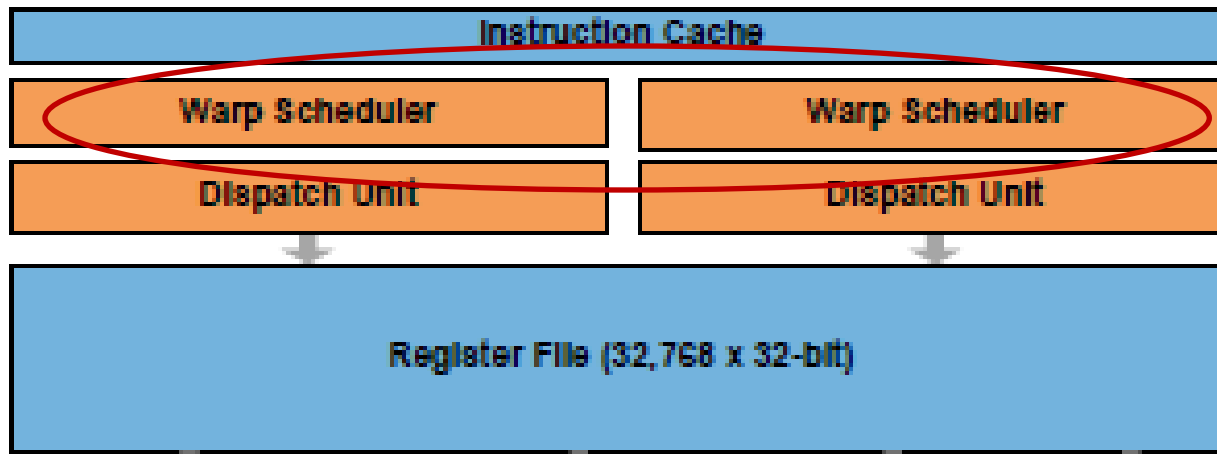
Arquitectura Fermi

- Algunos de los cambios más importantes son:
 - Los SMs están organizados en dos bloques de 16 cores cada uno.
 - Tienen 4 SFUs lo que permite que en ocho ciclos de reloj ejecute un warp.
 - El pipeline de SFUs está desacoplado de la dispatch unit por lo que se pueden despachar instrucciones a otras unidades mientras las SFUs están ocupadas.
 - Aumento significativo en la performance de punto flotante de doble precisión (relación 2x con simple precisión).

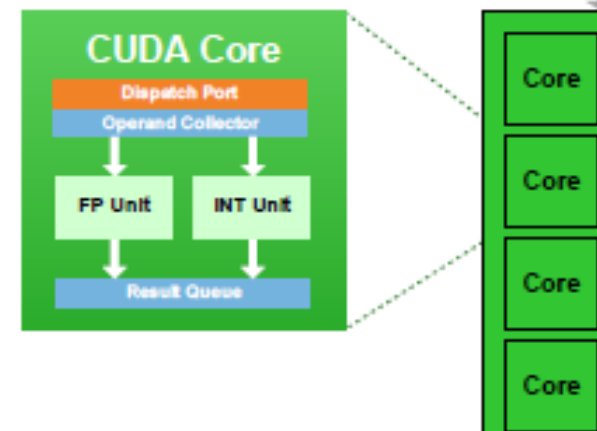


Arquitectura Fermi

- Algunos de los cambios más importantes son:
 - Cada SM tiene dos planificadores de warps (atienden dos warps a la vez, uno para bloque de cores) y una memoria en el chip de 64 KB.

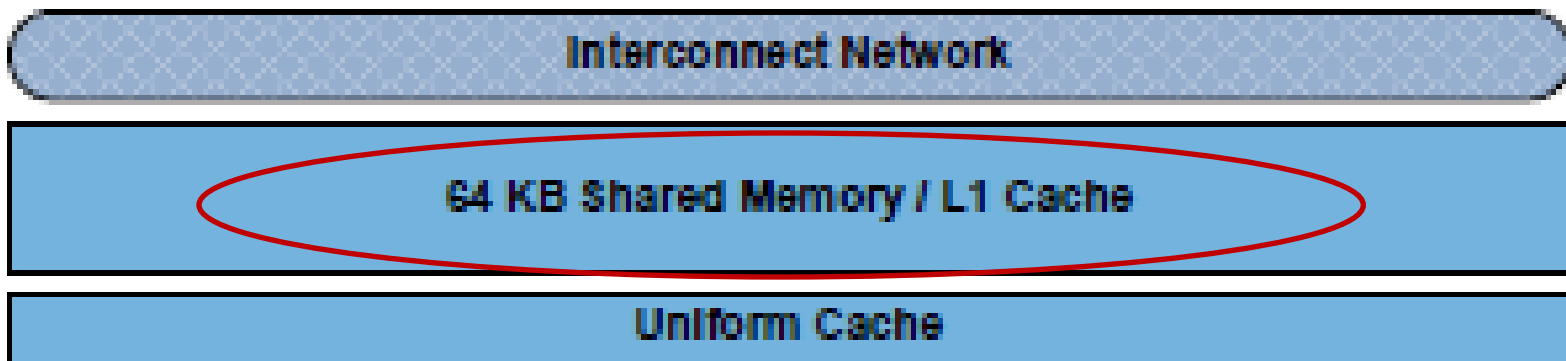


- Cada core se compone de una unidad para el procesamiento de punto flotante y una ALU.



Arquitectura Fermi

- Algunos de los cambios más importantes son:
 - La memoria del chip funciona como un caché de primer nivel para la memoria global y una parte como memoria compartida.
 - Puede ser dividida 16K/48K o 48K/16K entre caché y memoria compartida.

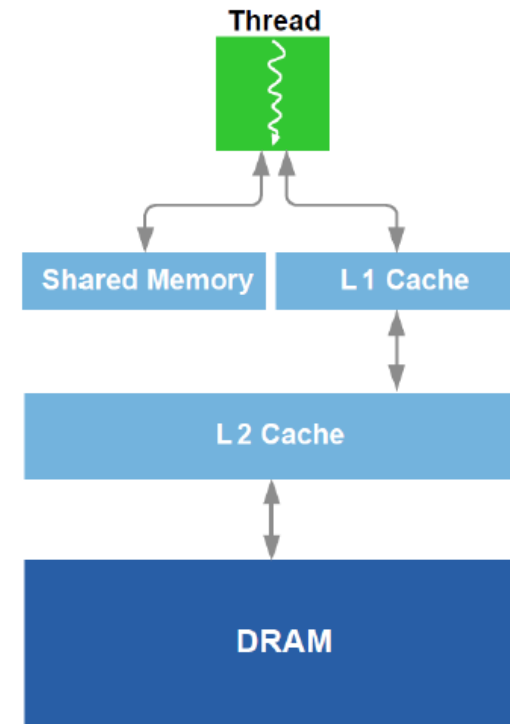


Arquitectura Fermi

- Algunos de los cambios más importantes son:
 - También se incorpora un caché de segundo nivel de 768 KB para el acceso a la memoria global.



Fermi Memory Hierarchy



- Se incorporan mecanismos para la detección y corrección de errores en el acceso a datos.

Arquitectura Kepler

Arquitectura Kepler

- En 2012 se lanza la tercera arquitectura de CUDA, conocida con el nombre de Kepler.
- Mejora significativa en la performance.
- Sin embargo, el foco está puesto en reducir el consumo de energía.
- El objetivo más importante del diseño de la arquitectura Kepler es obtener una mejor performance por watt.

Arquitectura Kepler

- **Algunos cambios importantes son:**
 - Los nuevos Streaming Multiprocessor se llaman SMX y funcionan a la frecuencia del reloj de los gráficos. Anteriormente los SMs funcionaban a la frecuencia de los shaders que eran aproximadamente 2x de los gráficos.
 - La reducción en la frecuencia es clave para la disminución en el consumo de energía:
 - Fermi GF110: Shader clock 1544 Mhz - Graphics clock 772 Mhz.
 - Kepler GK104: Graphics clock 1006 Mhz.
 - Pero entonces ¿cómo se logra mejorar la performance?
 - Aumentando la cantidad de cores!!!
 - Fermi GF110: 512 CUDA cores
 - Kepler GK104: 1536 CUDA cores

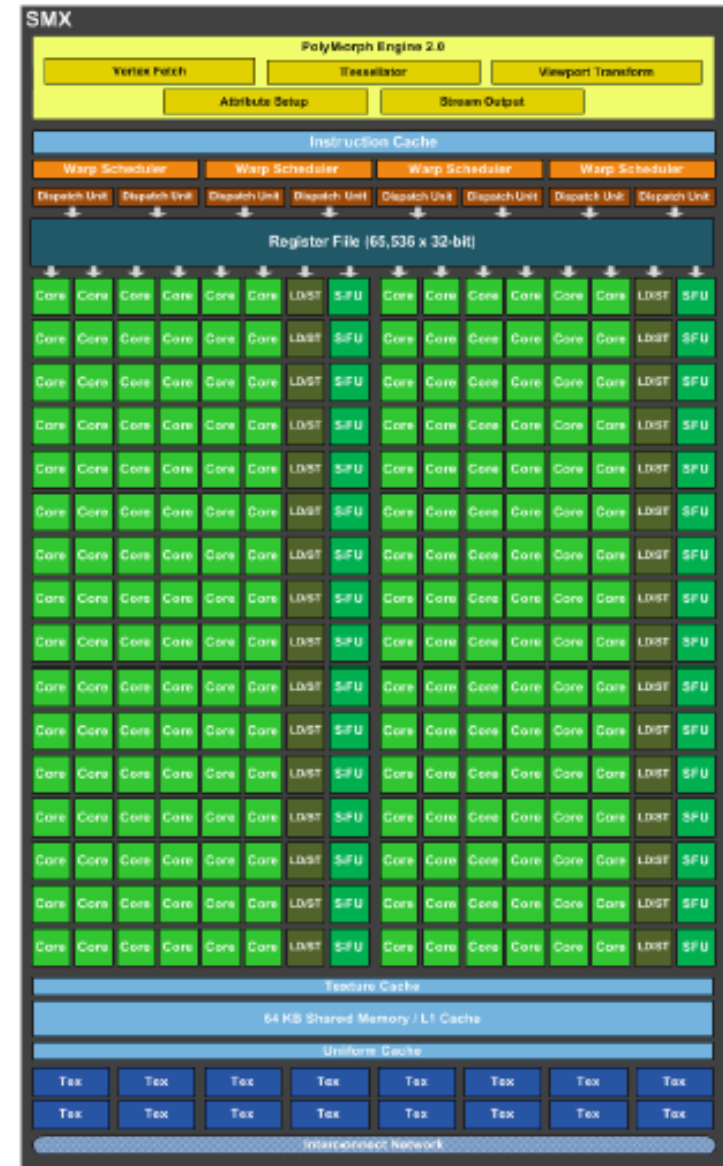
Arquitectura Kepler

- Algunos cambios importantes son:
 - Tiene 4 GPCs (Graphics Processing Clusters) compuestos por 2 SMXs. En total son 8 SMXs.
 - Utiliza PCI Express 3.0 para la comunicación con el host. Hasta la arquitectura Fermi se usaba PCI Express 2.0.



Arquitectura Kepler

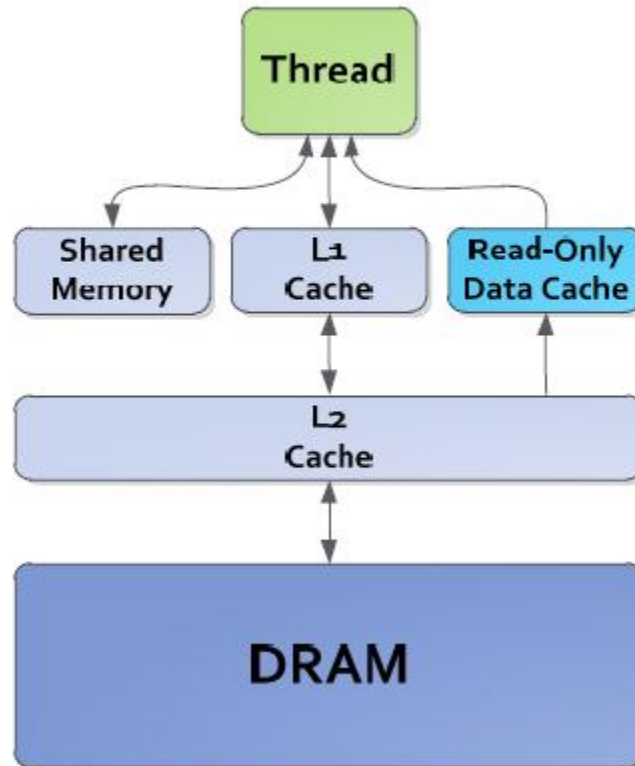
- Algunos cambios importantes son:
 - Los SMXs están compuestos por 192 CUDA cores. En total son 1536 CUDA cores.
 - Tienen 32 SFUs por SMXs. En total son 256 SFUs.
 - Tienen 4 Warp Schedulers aunque la cantidad total se mantiene en 32.



Arquitectura Kepler

- Algunos cambios importantes son:
 - Se incorpora un caché de sólo lectura de 48K.

Kepler Memory Hierarchy



Arquitectura Maxwell

Arquitectura Maxwell

- Sobre fines de 2014 se lanza la cuarta arquitectura de CUDA, conocida con el nombre de Maxwell.
- Salto en la eficiencia energética reduciendo el consumo pero aumentando la performance.
- La performance por watt es casi el doble que en la arquitectura Kepler.



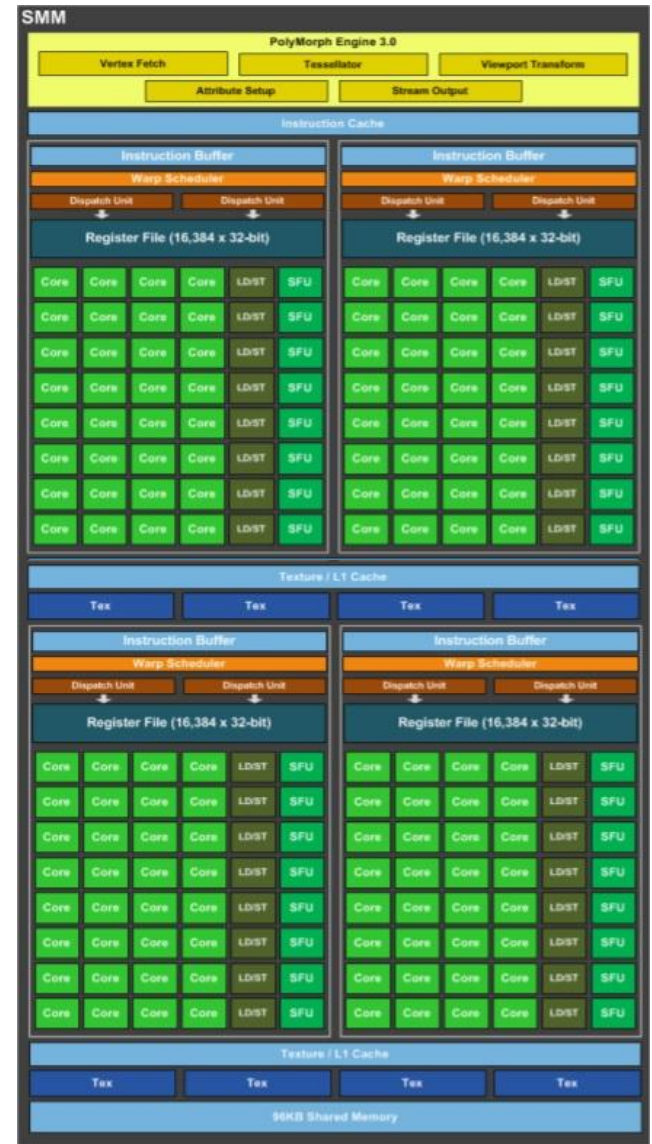
Arquitectura Maxwell

- Se compone de:
 - 4 GPCs
 - 16 multiprocesadores
 - 4 controladores de memoria
- Cada multiprocesador tiene 128 CUDA cores.
- 2048 CUDA cores en total.



Arquitectura Maxwell

- Los multiprocesadores están particionados en 4 bloques, cada uno con sus propios recursos independientes para la planificación de hilos y el búfer de instrucciones.
- Cada multiprocesador contiene 4 warp schedulers.
- Cada warp scheduler es capaz de despachar dos instrucciones por warp en cada ciclo de reloj.



Arquitectura Pascal

Arquitectura Pascal

- En abril 2016 se lanza la quinta arquitectura de CUDA, conocida con el nombre de Pascal.
- La arquitectura completa tiene 60 multiprocesadores!!!



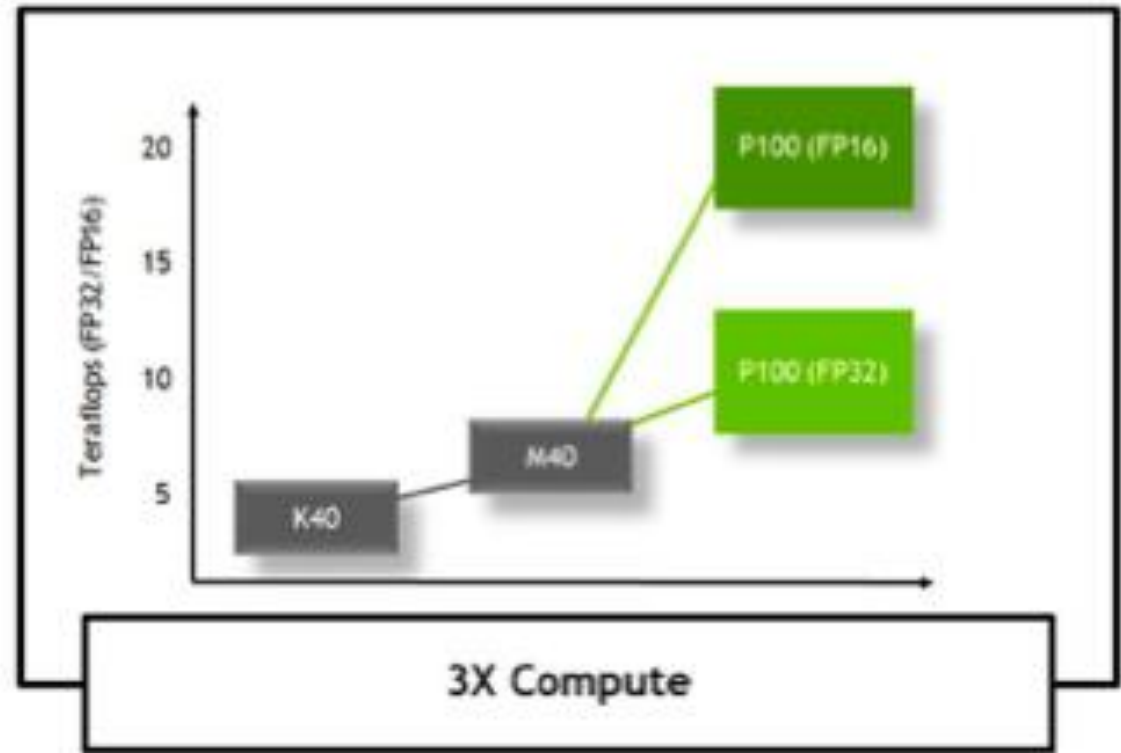
Arquitectura Pascal

- **Ejemplo de esta arquitectura es la Tesla P100.**
- **Existe un Servidor diseñado especialmente para Deep Learning que combina 8 Tesla P100 (Servidor DGX-1).**
- **Es un salto cualitativo importante**
- **Tiene varias mejoras con respecto a las arquitecturas previas.**

Arquitectura Pascal

- Mejoras en hardware
- Mejora comunicación CPU – GPU:
 - NVLink
 - mayor bandwidth
- FP16 real:
 - mejora importante en el desempeño (si puede usar FP16)

Desempeño teórico P100



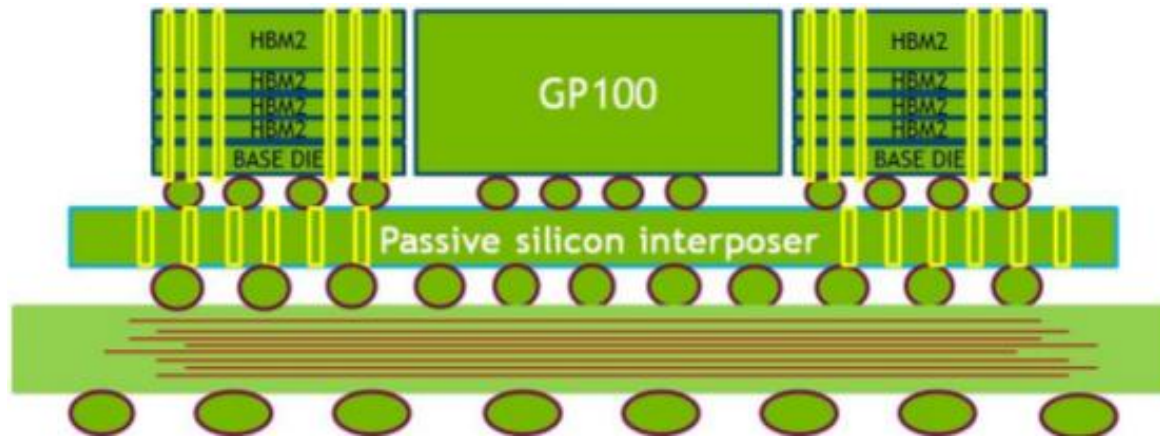
- 5.3 TFLOPS en doble precisión floating point (FP64)
- 10.6 TFLOPS en simple precisión (FP32)
- 21.2 TFLOPS en half-precisión (FP16)

NVLink

- **Tecnología de Nvidia**
- **Ofrece transferencias de datos GPU--GPU de hasta 160 Gigabytes/segundos de bandwidth.**
- **En algunos equipos CPU-GPU (p.ej. IBM power 8).**
- **5x cuando se lo compara con el bandwidth de PCIe Gen 3 x16.**
- **Comparable en velocidad con InfiniBand o 100 Gb Ethernet (alta velocidad).**

Memoria

HBM2 High-Speed GPU Memory Architecture



- **Stacked RAM: tecnología nueva 3D**
- **Tesla P100 primera GPU que soporta memoria HBM2.**
- **3x en el bandwidth de memoria sobre la Maxwell GM200 GPU.**
- **Como la memoria HBM2 es stacked permite mejor diseño de HW.**

Arquitectura Pascal

La GP100 está compuesta por (hasta):

- **6 array of Graphics Processing Clusters (GPCs).**
- **30 Texture Processing Clusters (TPCs).**
- **60 Streaming Multiprocessors (SMs):**
 - **Cada SM tiene 64 CUDA Cores y 4 Texture Units**

En total 3840 simple precisión CUDA Cores y 240 texture units.

- **Cada controlador de memoria tiene 512 KB de caché L2.**
- **Cada HBM2 DRAM tiene 2 controladores.**

En total ofrece 4096 KB de caché L2 (512 x 8).

Arquitectura Pascal

Mejora importante en doble precisión:



Arquitectura Pascal

Mejora importante en doble precisión:

Cada SM ofrece 32 CUDA Cores de doble precisión.

- **La mitad que FP32**
- **En total tiene 1920 FP64 CUDA Cores**

El ratio entre SP vs DP es 2:1

- **Kepler GK110 tiene un ratio 3:1 entre SP y DP.**

Soporte IEEE 754-2008 completo, incluso fused multiply-add (FMA) y desnormalizados.

Arquitectura Pascal

Soporte para FP16

- Especialmente útil para Deep Learning
 - Es de las pocas técnicas que no necesitan más precisión.
- Obviamente se reduce el uso de memoria (x2 vs FP32 y x4 vs FP64).
- También disminuyen los accesos a memoria.
- En GP100 se pueden realizar 2 operaciones de FP16 por operación en FP32.

Servidor DGX-1

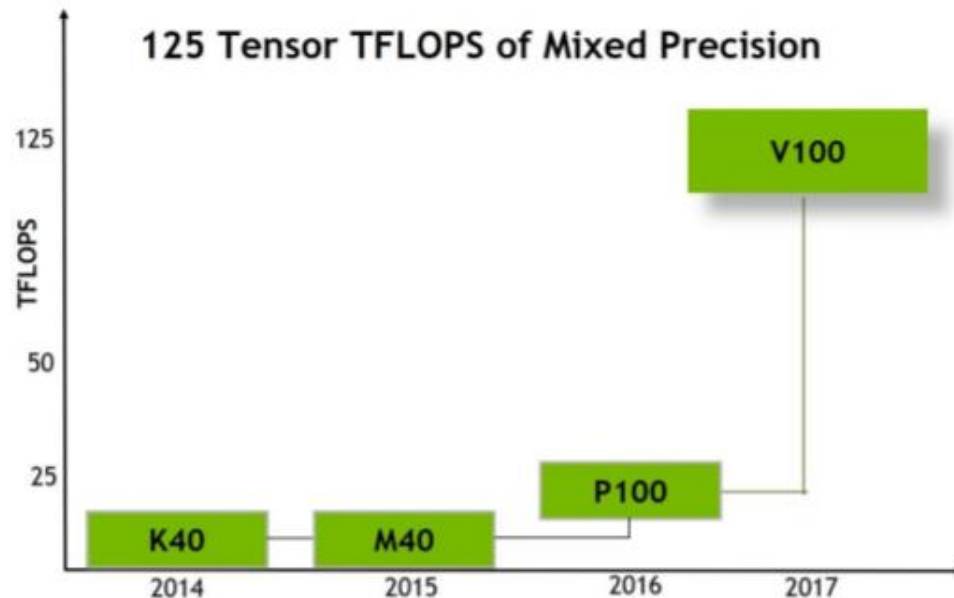
Table 4. NVIDIA DGX-1 System Specifications

Specification	Value
GPUs	8x Tesla P100 GPUs
TFLOPS	170 (GPU FP16) + 3 (CPU FP32)
GPU Memory	16 GB per GPU / 128 GB per DGX-1 Node
CPU	Dual 20-core Intel® Xeon® E5-2698 v4 2.2 GHz
NVIDIA CUDA Cores	28,672
System Memory	512 GB 2133 MHz DDR4 LRDIMM
Storage	4x 1.92TB SSD RAID 0
Network	Dual 10 GbE, 4 IB EDR
System Weight	134 lbs
System Dimensions	866 D x 444 W x 131 H (mm)
Packing Dimensions	1180 D x 730 W x 284 H (mm)
Power	3200W (Max). Four 1600W load-balancing power supplies (3+1 redundant), 200-240V (ac), 10A
Operating Temperature Range	10 - 35°C

Arquitectura Volta

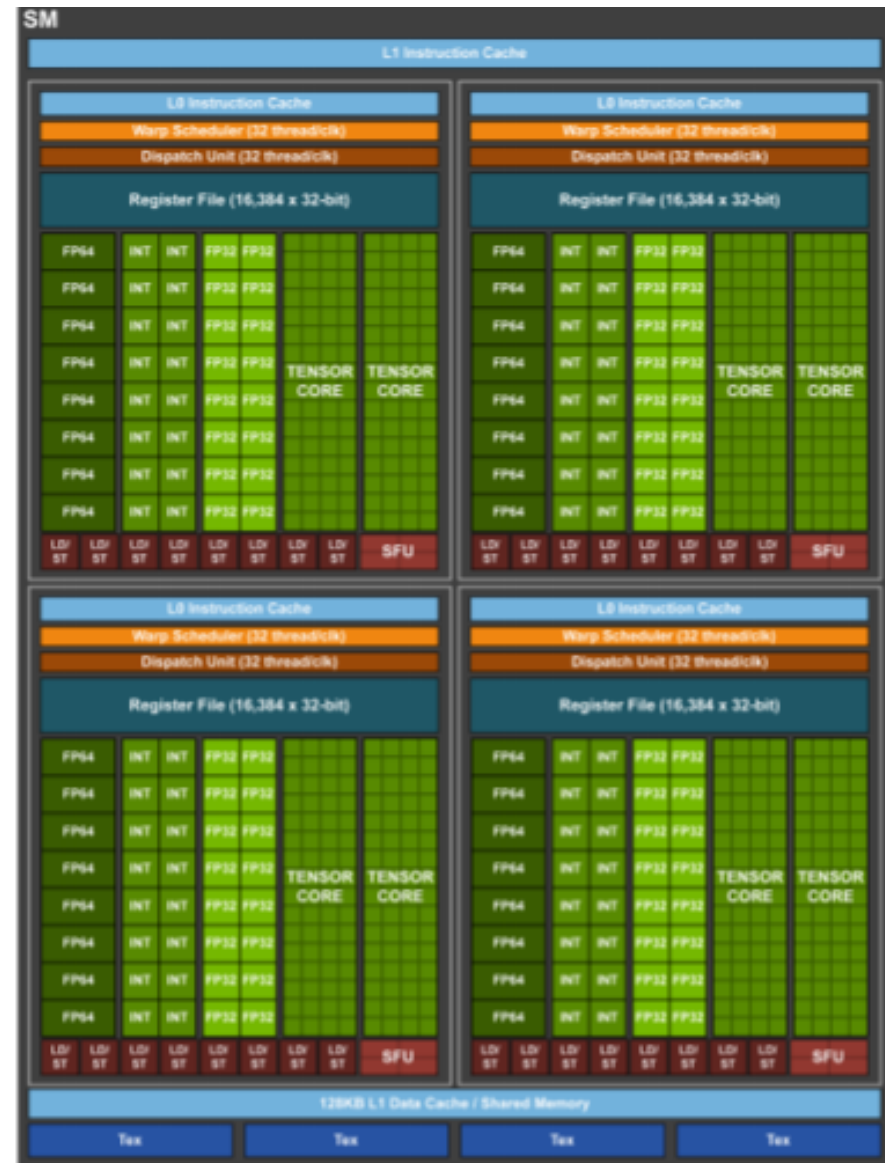
Arquitectura Volta

- En mayo 2017 se lanza la sexta arquitectura de CUDA, conocida con el nombre de Volta.
- 7.8 TFLOPS en doble precisión floating point (FP64)
- 15.7 TFLOPS en simple precisión (FP32)
- 125 TFLOPS en half-precisión (FP16)



Arquitectura Volta

- La novedad más importante es la inclusión de un nuevo tipo de unidad de cómputo, los tensor cores.
- Además, se separan las unidades de FP32 de las de INT.
- Permite la ejecución simultánea de operaciones enteras y FP32.



Arquitectura Volta

- **Los tensor cores:**
 - Cada Tensor Core realiza 64 fused multiply–add (FMA) de punto flotante por ciclo de reloj
 - Hay 8 en cada multiprocesador y pueden hacer 512 operaciones FMA (o 1024 operaciones individuales).
- La V100 tiene una performance 1.8x superior que la P100 en la multiplicación de matrices de simple precision.
- Usando los tensor cores, que trabajan con entradas en half-precision (FP16) y acumulan en FP32, la mejora es de 9x.

Arquitectura Turing

Arquitectura Turing

- A fines de 2018 se lanza la séptima arquitectura de CUDA, conocida con el nombre de Turing.
- La GeForce RTX 2080 Ti es de esta arquitectura:
 - 4352 CUDA cores
 - Memoria 11 GB, ancho de banda 616 GB/s (GDDR6)
 - 0.367 TFLOPS en doble precisión floating point (FP64)
 - 13.4 TFLOPS en simple precisión (FP32)
 - 26.9 TFLOPS en half-precisión (FP16)
 - 107.6 Tensor TFLOPS with FP16 Accumulate

Arquitectura Turing

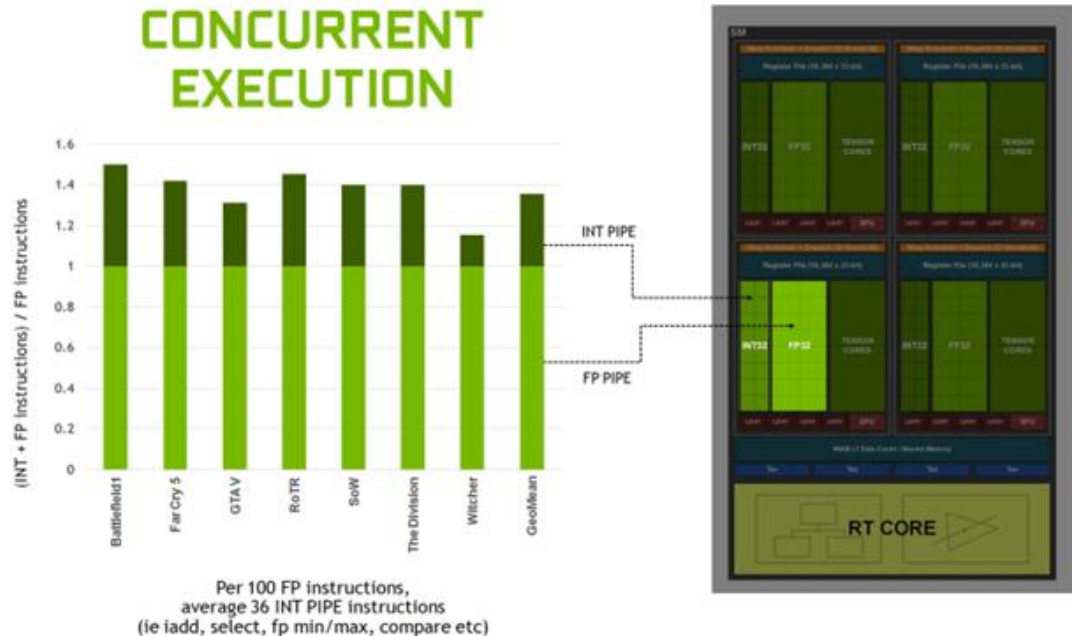


Note: The TU102 GPU also features 144 FP64 units (two per SM), which are not depicted in this diagram. The FP64 TFLOP rate is 1/32nd the TFLOP rate of FP32 operations. The small number of FP64 hardware units are included to ensure any programs with FP64 code operates correctly.

GPU Features	GTX 1080Ti	RTX 2080 Ti
Architecture	Pascal	Turing
GPCs	6	6
TPCs	28	34
SMs	28	68
CUDA Cores / SM	128	64
CUDA Cores / GPU	3584	4352
Tensor Cores / SM	NA	8
Tensor Cores / GPU	NA	544
RT Cores	NA	68
GPU Base Clock MHz (Reference / Founders Edition)	1480 / 1480	1350 / 1350

Arquitectura Turing

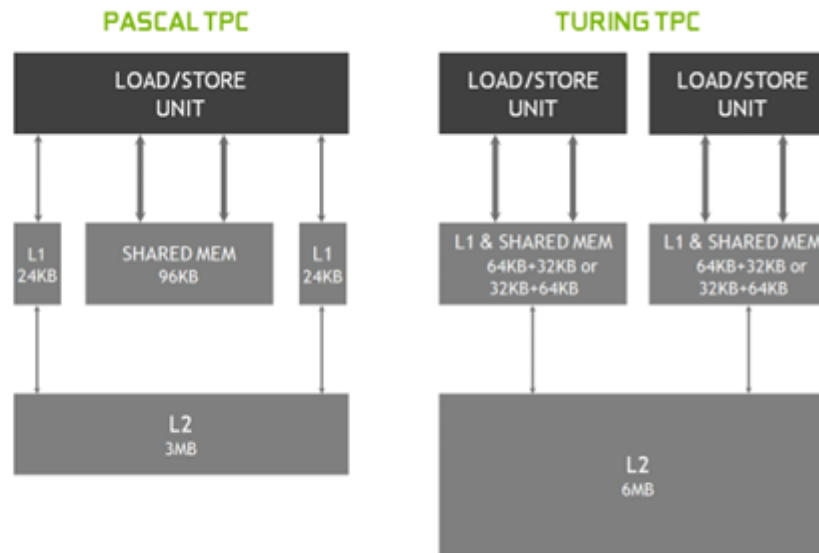
- Algunas mejoras:
 - Se incorporan los Ray Tracing cores.
 - Ejecución concurrente de instrucciones de punto flotante y de enteros.



Arquitectura Turing

- Algunas mejoras:
 - Nuevo diseño de la shared memory

2x L1 Bandwidth
Lower L1 Hit Latency
Up to 2.7x L1 Capacity
2x L2 Capacity



Arquitectura Turing

- Algunas mejoras:
 - Los tensor cores incorporan manejo de int8 e int4 para dar soporte a quantization (discretización).

GPU Features	GTX 1080Ti	RTX 2080 Ti
Peak FP16 Tensor TFLOPS with FP16 Accumulate* (Reference/Founders Edition)	NA	107.6 / 113.8
Peak FP16 Tensor TFLOPS with FP32 Accumulate* (Reference/Founders Edition)	NA	53.8 / 56.9
Peak INT8 Tensor TOPS* (Reference/Founders Edition)	NA	215.2 / 227.7
Peak INT4 Tensor TOPS* (Reference/Founders Edition)	NA	430.3 / 455.4

Compute Capabilities

Comparando Arquitecturas

	G80	Fermi	Kepler
Transistores	681 mill	3.0 bill	3.54 bill
CUDA cores	128	512	1536
Core clock	1350 Mhz	1544 Mhz	1006 Mhz
Thermal Design Power	155 W	244 W	195 W
Theorical GFLOPS/s	518	1581	3090
GFLOPS/W	3.34	6.48	15.85
Memory Bandwidth	86.4 GB/sec	192.4 GB/sec	192.26 GB/sec

Arquitectura CUDA – Kepler

	Maxwell	Pascal	Volta	Turing (T40)
Transistores	5.2 bill	7.2 bill	21.1 bill	18.6 bill
CUDA cores	2048	2560	5120	4608
Core clock	1126 Mhz	1607 Mhz	1200 Mhz	1305 Mhz
Thermal Design Power	165 W	180 W	250 W	250 W
Theorical GFLOPS/s	4612	8228	13800	14380
GFLOPS/W	27.95	45.71	55.2	57.5
Memory Bandwidth (GB/sec)	224	320	652.8	624

Compute Capabilities

- Para hacer uso óptimo de las GPUs, es necesario conocer diferentes características de la tarjeta.
- Nvidia utiliza un formato estandarizado para especificar estas características denominado compute capabilities.
- La categorización incluye dos números.
- Los cambios en la primera cifra implican cambios de generación, mientras que en la segunda implica una revisión.
- Las primeras GPUs de CUDA eran compute capability 1.0.
- Las últimas GPUs son:
 - arquitectura Ampere: compute capabilities 8.0 a 8.7
 - arquitectura Ada Lovelace: compute capabilities 8.9
 - arquitectura Hopper: compute capabilities 9.0
 - arquitectura Blackwell: compute capabilities 10.0, 10.3, 11.0, 12.0 y 12.1.

Compute Capabilities

Compute Capability Version	Chip de la GPU	Modelos de tarjetas
1.0	G80, G92, GF106	GeForce 8800GTX, 9800GT, Tesla C/D/S870, GT420, GT430, GT440
2.0	GF100, GF110	GeForce GTX 465, 470, 480, 570, 580, 590, Tesla C2050, C2070, S/M2050/70
5.2	GM200, GM204, GM206	GeForce GTX Titan X, 980 y 980 Ti, Tesla M40
6.1	GP100, GP102, GP104, GP106, GP107, GP108	Nvidia Titan X, GeForce GTX 1080 y 1080 Ti, Tesla P40, Tesla P100
7.0	GV100	Nvidia Titan V, Tesla V100
7.5	TU102, TU104	GeForce RTX 2080 Ti, Tesla T4

- Han existido las compute capabilities: 1.0, 1.1, 1.2, 1.3, 2.0, 2.1, 3.0, 3.2, 3.5, 3.7, 5.0, 5.2, 5.3, 6.0, 6.1, 6.2, 7.0, 7.2, 7.5, 8.0, 8.6, 8.7, 8.9, 9.0, 10.0, 10.3, 11.0, 12.0 y 12.1.

Compute Capabilities

	1.0	2.x	3.7	5.2	7.0	7.5
Tamaño de warp	32	32	32	32	32	32
Warps residentes por MP	24	48	64	64	64	32
Hilos residentes por MP	768	1536	2048	2048	2048	1024
Registros por MP	8 K	32 K	128 K	64 K	64 K	64 K
Dimensión máxima del grid	2	3	3	3	3	3
Máximo de instrucciones por kernel	2 millones	512 millones				